

```
<CheckBox android:checked="@={user.adult}"
    .. />
<EditText android:id="@+id/firstName"
    android:visibility="@={user.adult ? View.
VISIBLE : View.INVISIBLE}"
    .. />
<EditText android:visibility=@{firstName.visibility}
    .. />
```

Implied event updates

In the above example, we used a variable `adult` from our model to store the value of the state of a checkbox and update the visibility of other views, but this can be done more easily by directly passing the result of the checked state to another expression. So now, modifying the code shown earlier, we get the following output:

```
<CheckBox android:id="@+id/checkbox"
    .. />
<EditText android:id="@+id/firstName"
    android:visibility="@={checkbox.checked ?
View.VISIBLE : View.INVISIBLE}"
    .. />
<EditText android:visibility=@{firstName.visibility}
    .. />
```

When the checkbox's checked property changes, the result is directly applied to the expression and the view's visibility changes accordingly.

Lambda expressions

Lambda expressions are a very cool feature of Java 8. Anyone who is familiar with Java 8 will agree. But as the Android platforms before Android Nougat had features up to Java 7 only, we unfortunately don't get to use them. But in data binding, the framework parses the expressions and it completely removes those expressions at compile time. We can put almost any feature in data binding and it will still work on all the older platforms that data binding supports. At a very basic level, you can imagine lambda expressions as a syntactic sugar for listener interfaces. Let's see how we can use lambda expressions in our layout files.

So let's suppose there is a method in our model, which we would like to call when the checkbox is checked; for example, showing a toast of the current state of the checkbox. To achieve this, we can directly write a lambda expression in our layout file which will call that method. In code, it looks like what's shown below:

```
<CheckBox android:id="@+id/checkbox"
    android:onClick="@{() -> user.
toastCheckboxState(context, checkbox)}"
    .. />
```

Here, the method `toastCheckboxState` is present in our model. It needs a checkbox to get the state and context to make a toast. Now, data binding has this cool feature that allows you to pass any view from the hierarchy to the method by referring to it from its ID. Another new feature is that you can get context in the layout file in a context named variable. Whenever you refer to the context variable, it will give you the context of the root view. But be aware that if you give a context name to anything else, such as a view's ID or a field in the model, it will override the context variable of the data binding framework and, instead, return the view or value from the model, respectively; so do try to avoid naming anything as context.

This is how lambda expressions can be used to put method calls easily in layout files. There is also another feature called method references, which works in a manner similar to lambda expressions, but behaves differently and has some different use cases. Readers are advised to explore this feature as an exercise as we're not covering it in the article, because we believe that most of the time lambda expressions will do the trick.

There are yet more features available in the data binding framework which we haven't covered. These include binding callbacks that give you control over when the data is bound with the UI, and writing our own data bound fields and synthetic events to make our own fields two-way data bound.

We advise you to start using data binding in your layouts because it reduces quite a bit of code. It also makes the code cleaner by dividing the logic between Java files and layout files, whereas previously, we had to do all of the logic in our Java code. It also increases the performance because it gets a reference to all the views with IDs at compile time. So we don't need to call `findViewById` for any view (`findViewById` is a heavy call as it scans the whole view hierarchy to find our view). 

References

- 1] <https://developer.android.com/topic/libraries/data-binding/index.html>
- 2] <https://github.com/google/android-ui-toolkit-demos/tree/master/DataBinding/DataBoundRecyclerViewView>
- 3] <https://www.youtube.com/watch?v=DAMMN7m3wLU>
- 4] <https://github.com/samvidmistry/data-binding-demo> - Link to demo used in article

By: Samvid Mistry and Prof. Prakash Patel

Samvid Mistry is an Android developer at SculptSoft. He has a diploma in computer engineering from Nirma University, Ahmedabad. Email: mistrysamvid@gmail.com, <http://www.samvidinfotech.in>

Prof. Prakash Patel is working as Assistant Professor in the Information Technology department at Gandhingar Institute of Technology. Email: prakashpatel@yahoo.co.in